

軟體工程

第四課：系統塑模與統一架構 (System Modeling & Unified Architecture)

授課教師：薛念林 教授

資訊工程學系

逢甲大學

第四章：課程藍圖與核心架構

第一部分：塑模基礎與核心靜態模型

- **4.1 系統塑模基礎：** 塑模本質、四大核心視角、五大必備圖表與盲人摸象寓言。
- **4.2 統一塑模語言 (UML)：** 1990 年代方法論之戰、UML 三巨頭三位一體分工與 OMG 標準化。
- **4.3 功能塑模與使用案例：** 參與者目標、系統邊界、include 與 extend 關聯及規格書。
- **4.4 結構塑模與領域類別圖：** 領域實體、可見度符號、關聯重數與聚合/組合生命週期。

第二部分：互動時序、狀態機與 AI 輔助塑模

- **4.5 互動塑模與循序圖：** 訊息傳遞時序、生命線、啟動條與 BCE 穩健性架構模式。
- **4.6 行為塑模與有限狀態機：** 反應型系統、訂單生命週期狀態機、事件觸發、守衛條件與動作。
- **4.7 AI 輔助系統塑模：** 文字轉圖視覺副駕駛、PlantUML/Mermaid 宣告式語法與人機協同審查。
- **4.8 核心複習與統整：** 重點回顧、概念填空測驗與經典權威文獻。

4.1 系統塑模基礎

"A language that doesn't affect the way you think about programming is not worth knowing."

— Alan Perlis

”

什麼是系統塑模？ (What is System Modeling?)

"系統塑模是為系統開發抽象模型的過程，每個模型代表該系統不同的視角或觀點。"

— Ian Sommerville, *Software Engineering* (10th ed.)

”

- 系統塑模的核心工程原則：

- **抽象化 (Abstraction)**：隱藏不必要的實作細節，突顯關鍵的架構結構、資料流動與相依關係。
- **多重視角 (Multiple Perspectives)**：沒有任何單一圖表能解釋整個複雜軟體；不同利害關係人需要不同的視角。
- **溝通與驗證 (Communication & Verification)**：作為產品經理、系統架構師、開發者與測試工程師之間的通用視覺語言。
- **實作藍圖 (Blueprint for Implementation)**：提供正式的結構與行為規格，作為後續編寫程式碼與資料庫結構的精確藍圖。

盲人摸象與多重視角模型 (The Parable of the Elephant & Multiple Perspectives)

寓言啟示：盲人摸象與系統塑模的「多重視角」必要性

① 摸到象鼻：「像一條巨蟒 / 水管！」

🎯 互動視角 (Interaction Perspective)

- 關注：訊息傳遞順序、呼叫流向與動態協同
- 盲點：忽視靜態資料結構與內部持久化

對應 UML：循序圖 (Sequence Diagram)

② 摸到象腿：「像一根堅硬巨柱！」

🏠 結構視角 (Structural Perspective)

- 關注：實體資料屬性、關聯、組合與靜態骨幹
- 盲點：看不出系統隨時間演進的動態行為

對應 UML：類別圖 (Class Diagram)

③ 摸到象身：「像一面高大磚牆！」

🛡️ 外部視角 (External / Context)

- 關注：系統邊界、運行環境與外部第三方 API
- 盲點：無法得知內部核心邏輯如何拆解實現

對應 UML：環境圖 (Context Diagram)

④ 摸到象尾：「像一條擺動細繩！」

⚡ 行為視角 (Behavioral Perspective)

- 關注：業務流程流轉、外在事件刺激與狀態轉換
- 盲點：無法單獨反映靜態物件職責與系統邊界

對應 UML：活動圖 / 狀態圖 (Activity / State)

💡 結論：任何單一模型皆為「局部投影」；唯有綜合不同視角，才能完整還原軟體架構全貌！

系統塑模的 4 大核心視角 (4 Core Perspectives)

- **1. 外部視角 (External Perspective) :**
 - 塑模系統處在的運行環境、外部合作夥伴與邊界範圍。
 - 明確劃分系統邊界：哪些由內部團隊開發，哪些委託給外部第三方服務。
- **2. 互動視角 (Interaction Perspective) :**
 - 塑模外部參與者與系統之間，或是內部協同物件之間的動態溝通與訊息傳遞。
- **3. 結構視角 (Structural Perspective) :**
 - 塑模系統資料組織、類別、屬性、方法與關聯性的靜態架構，與執行時間先後無關。
- **4. 行為視角 (Behavioral Perspective) :**
 - 塑模系統的動態執行行為、業務流程步驟，以及系統針對外在事件所產生的反應式狀態轉換。

現代實務必備的 5 大 UML 核心圖表

圖表類型	所屬視角	動態 / 靜態	主要軟體工程職責
1. 環境圖 (Context Diagram)	外部視角	靜態邊界	劃定系統邊界與外部夥伴系統的相依關係
2. 活動圖 (Activity Diagram)	行為視角	動態流程	視覺化呈現連續業務工作流程與平行並行邏輯
3. 使用案例圖 (Use Case Diagram)	互動視角	靜態契約	界定使用者目標與系統功能範疇邊界
4. 循序圖 (Sequence Diagram)	互動視角	動態時間	沿著時間生命線追蹤物件之間的循序訊息傳遞
5. 類別圖 (Class Diagram)	結構視角	靜態骨幹	定義領域實體、資料屬性、操作方法與物件關聯
(加分必學) 狀態機圖 (State Diagram)	行為視角	動態反應	塑模離散狀態、事件觸發與反應式生命週期

觀念檢核測驗 1 (CCQ 1)

軟體架構師若欲定義「領域實體資料的靜態組織方式，以及類別之間的繼承、關聯與包含關係」（完全不隨執行時序變動），應採取哪一種塑模視角？

- **A.** 外部視角 (External Perspective)
- **B.** 互動視角 (Interaction Perspective)
- **C.** 結構視角 (Structural Perspective)
- **D.** 行為視角 (Behavioral Perspective)



4.2 統一塑模語言 (UML)

「標準化的視覺語法，將軟體工程從封閉的方法論孤島，淬煉為全球共通的工程學門。」 „

標準化的迫切需求：1990 年代「方法論之戰」

- 物件導向程式設計的崛起 (1980 年代末至 1990 年代初)：
 - 軟體產業從程序導向程式碼 (C, Pascal) 大規模轉移至物件導向典範 (C++, Smalltalk)。
 - 工程師迫切需要一套圖形符號來視覺化表達類別、物件與關聯。
- 百家爭鳴的「方法論之戰 (Method Wars)」時代：
 - 超過 50 種相互競爭的物件導向塑模記號充斥商業市場。
 - 各派大師激烈爭論：類別究竟該畫成雲朵、矩形還是橢圓？繼承到底該用空心三角、實心箭頭還是虛線？
 - 嚴重的產業割裂碎片化：各公司之間無法交換架構模型，CASE 塑模軟體彼此互不相容，工程師換工作就得被迫重新學習一套新符號。

統一與標準化：從 Rational 到 OMG

- **1994 年 – Rational 展開統一整合：**
 - Jim Rumbaugh 離開奇異 (GE) 加入 Rational Software 與 Grady Booch 攜手，將 Booch 方法與 OMT 融合成「統一方法」(Unified Method v0.8)。
- **1995 年 – 「三巨頭 (Three Amigos)」合體：**
 - Ivar Jacobson 加入 Rational，帶來革命性的使用案例 (Use Case) 與 OOSE 架構思維。
- **1997 年 – OMG 國際標準正式誕生：**
 - 提交給物件管理組織 (OMG)，於 1997 年 11 月獲全票通過，成為全球公認的 **UML 1.1** 國際標準。
- **2005 年 – UML 2.0 架構大改版：**
 - 擴充至 13 種（後續擴增為 14 種）圖表類型，並具備嚴謹的可執行元模型 (Metamodel)。

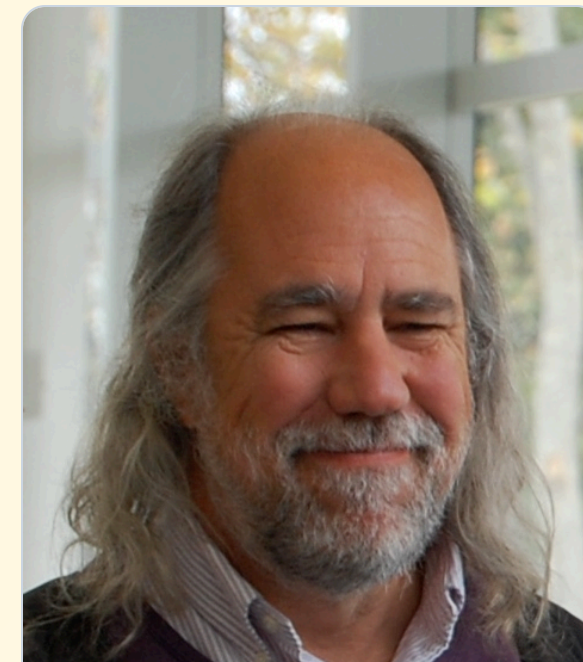


Unified Modeling Language
Object Management Group (OMG)

UML 奠基先驅：Grady Booch

- 角色與學術榮譽：
 - Rational Software 首席科學家、IBM 院士 (Fellow)、ACM 院士。
- 開創經典方法論：
 - **Booch** 方法與劃時代著作：《物件導向分析與設計》(Object-Oriented Analysis and Design with Applications)。
- 對 **UML** 的核心貢獻：
 - 極度專注於具體軟體設計、模組拆解、類別抽象化與架構模式。
 - 提倡為實作層級的物件結構與程式碼對應關係提供高度的視覺表現力。
- 著名軟體工程箴言：

"乾淨的程式碼讀起來，永遠就像是由深具責任心與熱忱的人所撰寫。"



Grady Booch

Rational Software / IBM Fellow

UML 奠基先驅：James Rumbaugh

- 角色與學術榮譽：

- 奇異公司 (GE) 全球研發中心軟體主任研究員、Rational Software、IBM。

- 開創經典方法論：

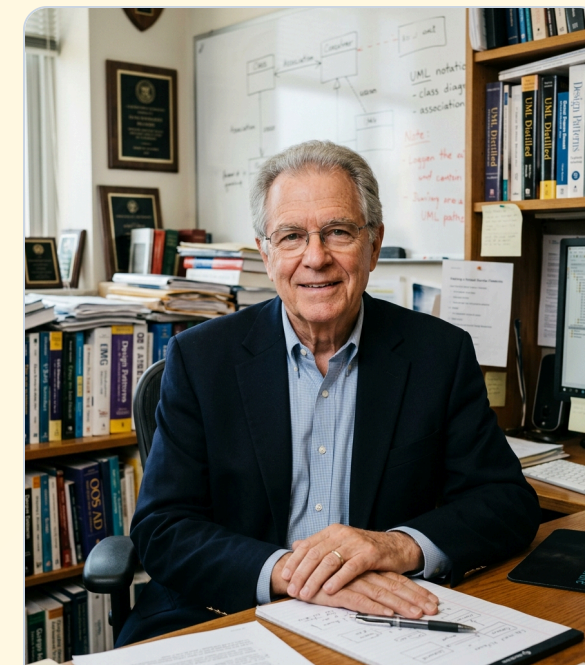
- 物件塑模技術 (OMT, Object Modeling Technique) 與專書：《物件導向塑模與設計》。

- 對 UML 的核心貢獻：

- 強調嚴謹的領域分析 (Domain Analysis)、語義資料塑模與實體關聯對應。
- 首創將靜態物件結構與 David Harel 的狀態圖 (Statecharts) 深度結合，用以塑模動態反應式系統。

- 著名軟體工程箴言：

“如果不透徹理解領域的真實本質，就絕不可能打造出卓越的軟體。”



James Rumbaugh

GE Research / Rational Software

UML 奠基先驅：Ivar Jacobson

- 角色與學術榮譽：
 - 愛立信 (Ericsson) 首席架構師、Objectory AB 創辦人、Rational Software、SEMAT 發起人。
- 開創經典方法論：
 - 物件導向軟體工程 (OOSE)。
- 對 UML 的核心貢獻：
 - 使用案例 (Use Cases) 的發明者 (1986)：革命性地將系統架構錨定在可衡量的使用者目標之上。
 - 提出經典的邊界-控制-實體 (BCE, Boundary-Control-Entity) 強韌性分析模式與元件化架構。
- 著名軟體工程箴言：

"一個沒有使用者的系統，就沒有任何存在的理由。塑模永遠要先從使用者的目標開始。"



Ivar Jacobson

Ericsson / Objectory / Rational

UML 三巨頭：軟體塑模的三位一體 (The Trinity of Modeling)

Ivar Jacobson

核心回答「Why」(目標 / 使用者)

- 使用案例驅動：外部參與者想達成什麼目標？
- **BCE** 強韌性分析：徹底分離 UI 邊界與核心資料實體。
- 元件契約規範：可重用子系統的架構邊界。

James Rumbaugh

核心回答「What」(領域與資料)

- 領域物件分析：現實世界存在哪些核心概念？
- 類別與資料綱要：結構化的實體關聯關係。
- 有限狀態機：反應式事件驅動的生命週期。

Grady Booch

核心回答「How」(設計與程式碼)

- 物件導向設計：類別之間如何精巧協同運作？
- 巨觀架構拆解：分層模組與相依性控制。
- 實作直譯對應：視覺模型與 OOP 程式碼直接對齊。

📌 架構整合精髓：Jacobson 捕捉系統存在的初衷目的 (**Why**)；Rumbaugh 塑模領域存在的資料本質 (**What**)；Booch 設計程式執行的實作架構 (**How**)。

”

觀念檢核測驗 2 (CCQ 2)

在被尊稱為「UML 三巨頭」的軟體工程大師中，哪一位以於 1986 年發明使用案例 (**Use Cases**)、將軟體架構直接錨定於使用者具體目標而聞名？

- **A.** Grady Booch
- **B.** James Rumbaugh
- **C.** Ivar Jacobson
- **D.** Martin Fowler



4.3 功能塑模與使用案例模型

"使用案例是系統與其參與者之間為了達成可衡量業務目標所簽署的行為契約。"

— Alistair Cockburn

”

使用案例塑模：核心定義與工程價值

「使用案例捕捉了外部參與者與系統之間的行為契約，旨在交付可衡量的業務價值。」

”

核心定義與抽象層次 (What It Is)

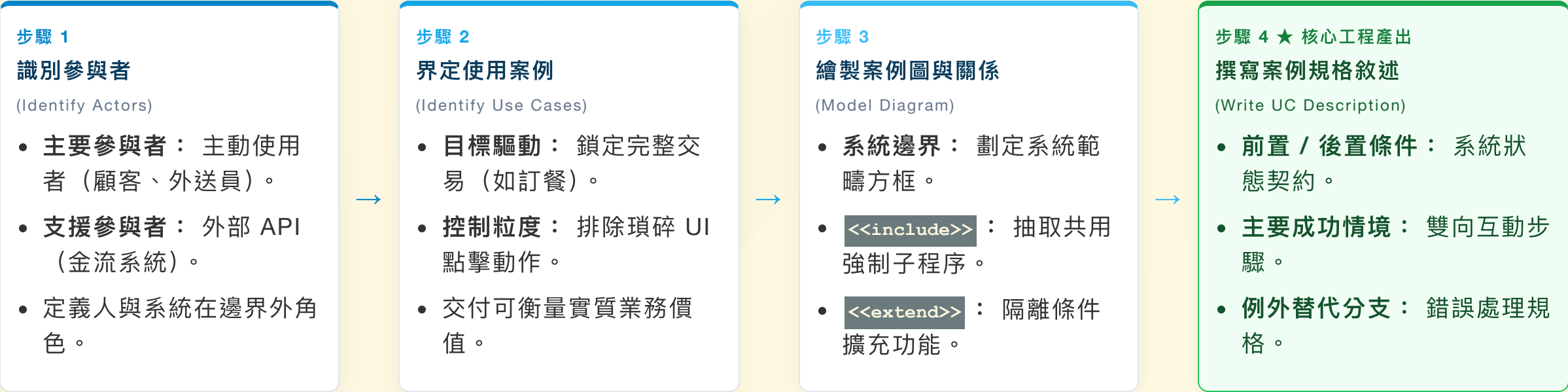
- **契約型功能範疇**：採取外部黑箱視角，精確界定系統為使用者提供「什麼 (What)」服務，不涉及內部程式細節。
- **目標導向的核心**：將軟體需求錨定在人類使用者或外部系統所欲達成的離散商業目標。
- **劃定系統邊界周界**：明確區隔哪些責任屬於軟體內部，哪些屬於外部使用者或第三方平台。

工程價值與防範風險 (Why It Is Important)

- **根除需求描述的模糊性**：取代散落各處、語意不清的文字條列，整合成目標一致的交易流程。
- **防止範疇蔓延 (Scope Creep)**：嚴謹的系統邊界方框，為衝刺開發與版本發布建立明確的交付承諾。
- **驗收測試 (UAT) 的直接依據**：主要成功情境與例外分支，可 100% 轉譯為端到端系統整合測試案例。

使用案例塑模：四步標準工程流程

「使用案例塑模絕不只是在圖上畫圈圈；結構化的使用案例規格敘述書，才是真正的行為契約。」

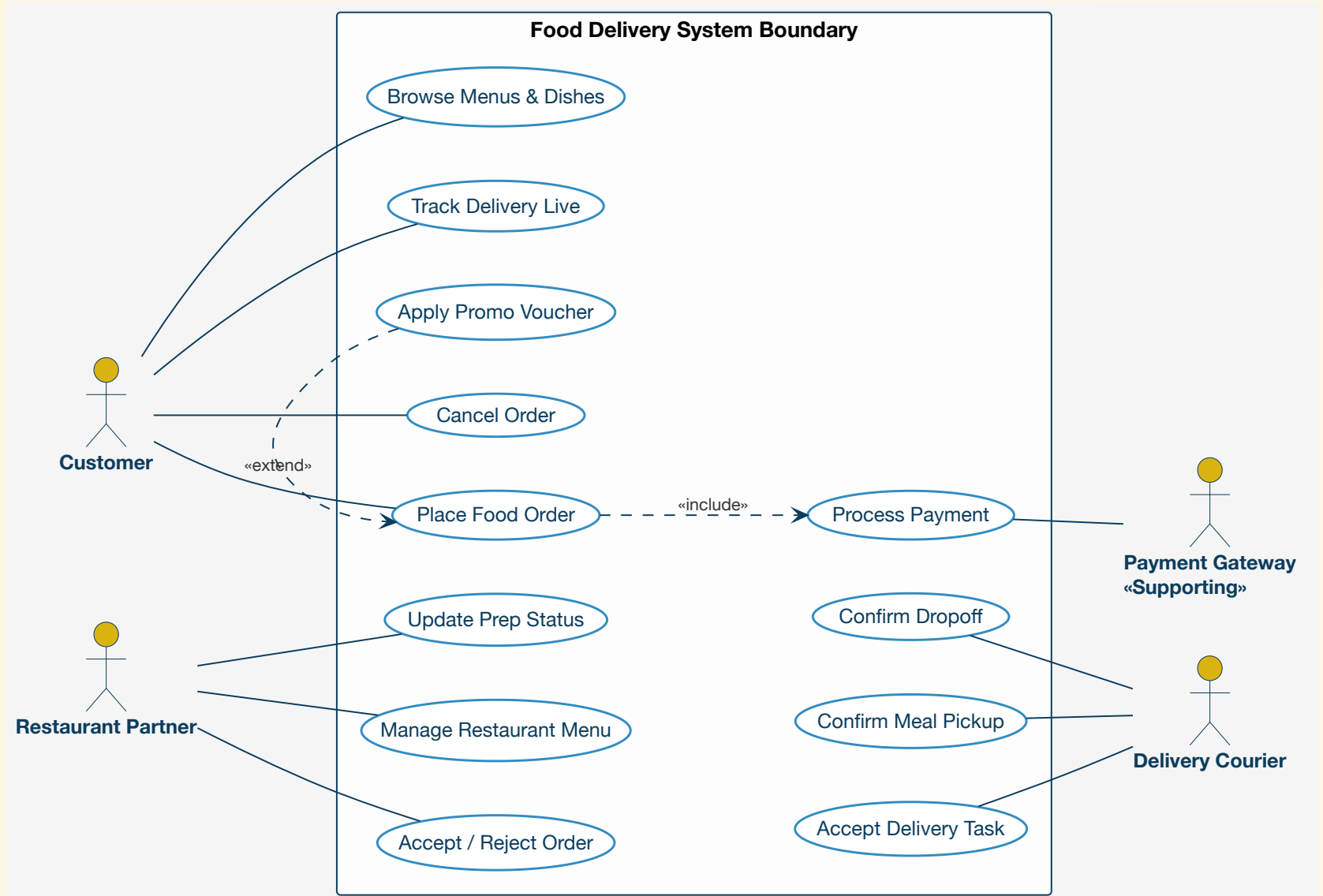


📌 **工程洞見：** 使用案例圖只是一張視覺化的目錄索引，使用案例規格敘述書 (**Use Case Description**) 才是指導工程師實作與品保人員驗收測試的真正契約。

步驟 1 & 2：識別參與者與界定目標導向的使用案例

- **主要參與者 vs. 支援參與者：**
 - **主要參與者 (Primary Actors，顧客、餐廳、外送員)：** 主動發起使用案例以達成個人業務目標的人類使用者。
 - **支援參與者 (Supporting Actors，第三方金流閘道)：** 由系統在背後呼叫以協助完成交易的外部服務系統。
- **系統邊界與參與關聯：**
 - 邊界矩形方框劃定「軟體系統內部」與「外部世界」的權責界線。
 - 實線關聯將參與者與其參與的案例橢圓相連。
- **案例粒度黃金法則 (Granularity Rule of Thumb)：**
 - 使用案例必須代表一項**完整的、能交付實質價值的業務交易**。
 - **反模式 (Anti-Pattern)：** 「點擊登入按鈕」、「輸入密碼」（此為微小 UI 動作，絕非使用案例！）。
 - **最佳實踐：** 「美食訂餐下單」、「管理餐廳菜單」（交付具體業務價值）。

步驟 3：美食外送平台完整使用案例圖



步驟 3：符號深解： <<include>> 與 <<extend>> 的關鍵差異

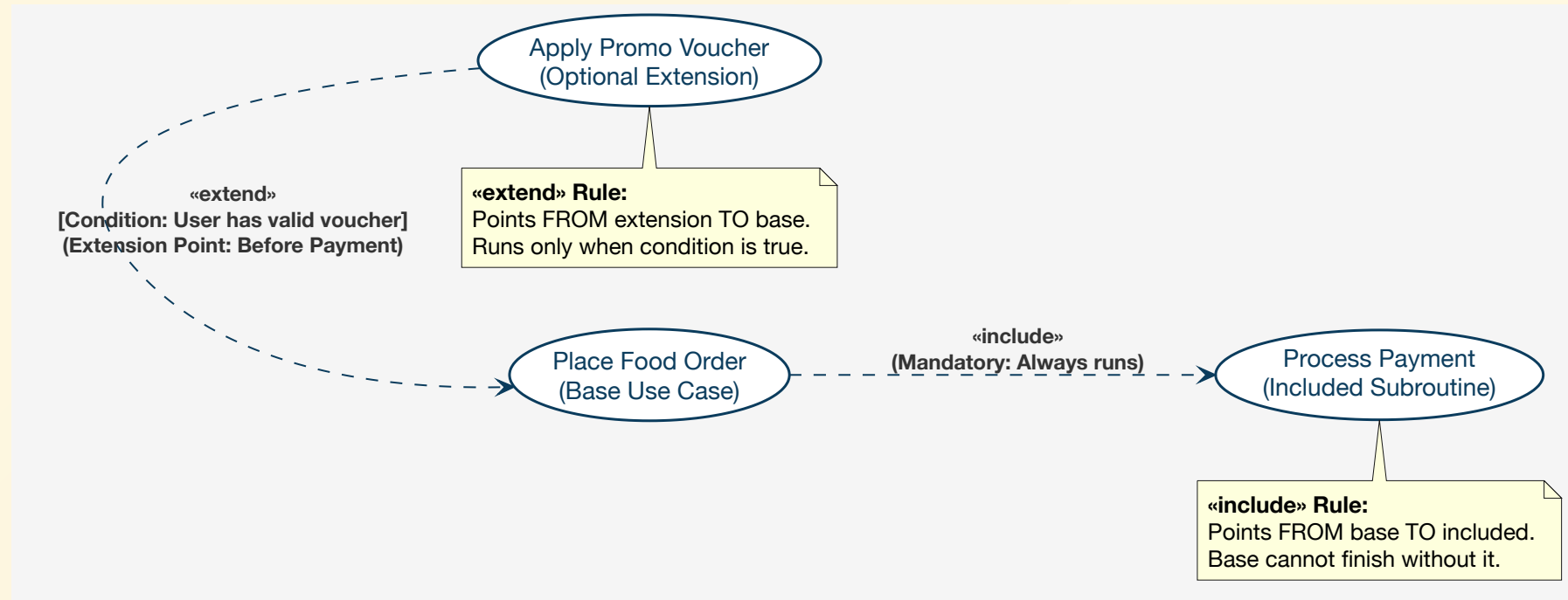
<<include>> 包含關係（強制執行）

- 語意本質：基礎案例若未執行被包含案例，則無法成功完成。
- 箭頭方向：由基礎案例指向被包含案例（基礎 ..> 被包含）。
- 工程目的：抽取多個案例共用的強制性子程序（例如：外送下單與會員訂閱皆需處理金流付款）。

<<extend>> 擴充關係（條件觸發）

- 語意本質：基礎案例本身已可獨立完成；僅在特定觸發條件滿足時才插入額外行為。
- 箭頭方向：由擴充案例指回基礎案例（擴充 ..> 基礎）。
- 工程目的：隔離非核心的可選邏輯（如套用優惠券），避免污染基礎情境。

步驟 3：Include 與 Extend 視覺語法語意解析



步驟 4：撰寫使用案例規格敘述書 (系統的行為契約)

「沒有書面規格敘述的使用案例，只是一個沒有故事的標題而已。」 — Alistair Cockburn

使用案例規格敘述書的核心結構

- **識別碼與名稱**：唯一編號 (如 UC-01) 與動賓片語標題 (如 美食訂餐下單)。
- **主要參與者與觸發事件**：誰發起此流程？何種商業事件促成互動開始？
- **前置條件 (Preconditions)**：流程開始前系統必須滿足的真值狀態 (如：顧客已認證、餐廳營業中)。
- **後置條件 (Postconditions)**：成功完成後系統保證達成的狀態 (如：訂單處於 PAID 狀態、廚房接獲通知)。

情境流程劃分與驗收角色

- **主要成功情境 (Main Flow)**：編號條列的雙向對話：使用者操作步驟 → 系統驗證與回饋。
- **替代與例外情境 (Alternative Flows)**：錯誤處理、卡號失效或可選擴充 (如 3a. 信用卡授權遭拒)。
- **驗收測試 (UAT) 的基石**：規格書中的每一條例外分支，直接對應至一組端到端自動化測試案例！
- **軟體架構的腳本**：主要成功情境直接作為循序圖 (4.5) 物件互動的設計腳本。

步驟 4：結構化使用案例規格書：美食訂餐下單 (UC-01)

規格書欄位	具體技術說明與規格定義
案例代號與名稱	UC-01: 美食訂餐下單 (Place Food Order)
主要參與者	顧客 (已註冊並通過認證之 FoodieGo App 會員)
前置條件 (Pre)	顧客處於已登入狀態；購物車內至少有 1 項有效餐點，且該餐廳處於營業接單狀態。
後置條件 (Post)	訂單建立並標記為 PAID 狀態；通知廚房接單；外送排程佇列已建立；收據寄發顧客。
主要成功情境 (Main Flow)	1. 顧客於購物車結帳頁面點擊「確認送出訂單」。 2. 系統進行即時庫存校驗，並精算餐點小計、外送費與應付總額。 3. 系統執行 <<include>> UC-02: 處理金流付款，向外部第三方金流驗證扣款。 4. 系統寫入持久化資料庫，產生唯一訂單編號 orderId 。 5. 系統向餐廳端管理儀表板推播即時訂單製備通知。 6. 系統建立 GPS 外送追蹤通訊工作階段，並回傳預估抵達時間。
擴充例外情境 (Extensions)	3a. 信用卡扣款授權遭拒： 3a1. 系統即時提示顧客卡號失效，訂單暫存保留為 DRAFT 待付狀態。 4a. <<extend>> 套用促銷優惠券： 4a1. 顧客於結帳前輸入有效代碼，系統於步驟 3 扣款前扣抵折扣金額。

使用案例模型：與其他 UML 模型的協同關係

「使用案例規格敘述書是軟體開發的單一事實來源，驅動結構、動態與狀態模型的建立。」

需求單一事實來源

4.3 使用案例模型

定義外部黑箱行為契約與端到端價值情境。

- 參與者與邊界劃定
- 目標導向使用案例
- 結構化規格敘述書



4.4 領域類別模型 (結構骨幹)

名詞分析法

使用案例步驟中的領域名詞 (Customer, Order, Dish) 直接提取為領域實體類別、資料屬性與持久化關聯。



4.5 互動循序圖 (動態交互)

場景動態實現

使用案例的主要情境與例外分支，直接轉譯為循序圖的時間序列腳本，由 BCE 物件協同實踐該案例。



4.6 有限狀態機 (生命週期)

觸發事件與前置守衛

使用案例中的操作是激發實體狀態躍遷的外部事件 (如付款使訂單轉為 PAID)；狀態機狀態亦守衛使用案例前置條件。

觀念檢核測驗 3 (CCQ 3)

在美食外送使用案例模型中，為何「套用優惠折扣碼」使用 `<<extend>>` 指向「美食訂餐下單」，而「美食訂餐下單」卻使用 `<<include>>` 指向「處理信用卡扣款」？

- **A.** 折扣碼是每筆訂單強制必填的欄位；而扣款則是自由選填的項目。
- **B.** 折扣碼為特定條件下的選用行為；而扣款則是結帳必定執行的共用程序。
- **C.** 折扣碼由次要參與者發起執行；而扣款則由主要參與者獨立完成。
- **D.** 折扣碼代表類別層級的繼承關係；而扣款代表物件層級的組合關係。



4.4 結構塑模與領域類別圖

"Classes are the static building blocks; objects are the living runtime instances."

(類別是系統的靜態積木；物件是運作時的鮮活實例。)

”

領域類別塑模：核心定義與工程價值

「類別圖定義了物件導向系統的靜態架構骨幹、領域實體關聯與資料封裝防線。」

”

何謂類別模型 (Definition & Abstraction)

- **靜態結構藍圖**：一種完全獨立於執行時序的模型，定義軟體型別、內部封裝狀態（屬性）與操作責任（方法）。
- **領域實體映射**：將真實世界問題領域的商業概念、實體與規則，精準轉化為物件導向軟體的抽象表示。
- **結構關聯分類學**：形式化定義關聯 (Association)、聚合 (◇)、組合 (◆) 與一般化繼承 (Generalization)。

為何至關重要 (Engineering Value)

- **程式碼與資料庫的直接映射**：標準三格矩形（類別名稱、屬性、方法）可 1:1 直接轉換為 OOP 原始碼與關聯式資料表架構。
- **精確控制物件生命週期耦合**：嚴格區分「同生共死」的強擁有關係（組合）與「彼此獨立」的弱擁有關係（聚合）。
- **資料封裝與語意完整性防線**：透過可見度符號（、、）保護內部狀態不變量，防範外部任意篡改。

領域類別塑模：四步標準工程流程

「領域塑模將真實世界中混亂的概念名詞，轉化為高內聚、低耦合的物件導向結構架構。」

步驟 1

名詞分析法

(Extract Nouns)

- 從使用案例規格敘述中提取候選實體。
- 排除短暫 UI 暫存值與無意義純量。



步驟 2

指派屬性與操作

(Attributes & Methods)

- 定義資料狀態與可見度 (+, -, #)。
- 依照單一職責指派核心業務操作。



步驟 3

建立關聯與重數

(Multiplicities)

- 繪製物件間的結構連線與角色名稱。
- 標註嚴格基數上下限 (1, 0..1, 1..*)。



步驟 4 ★ 生命週期控制

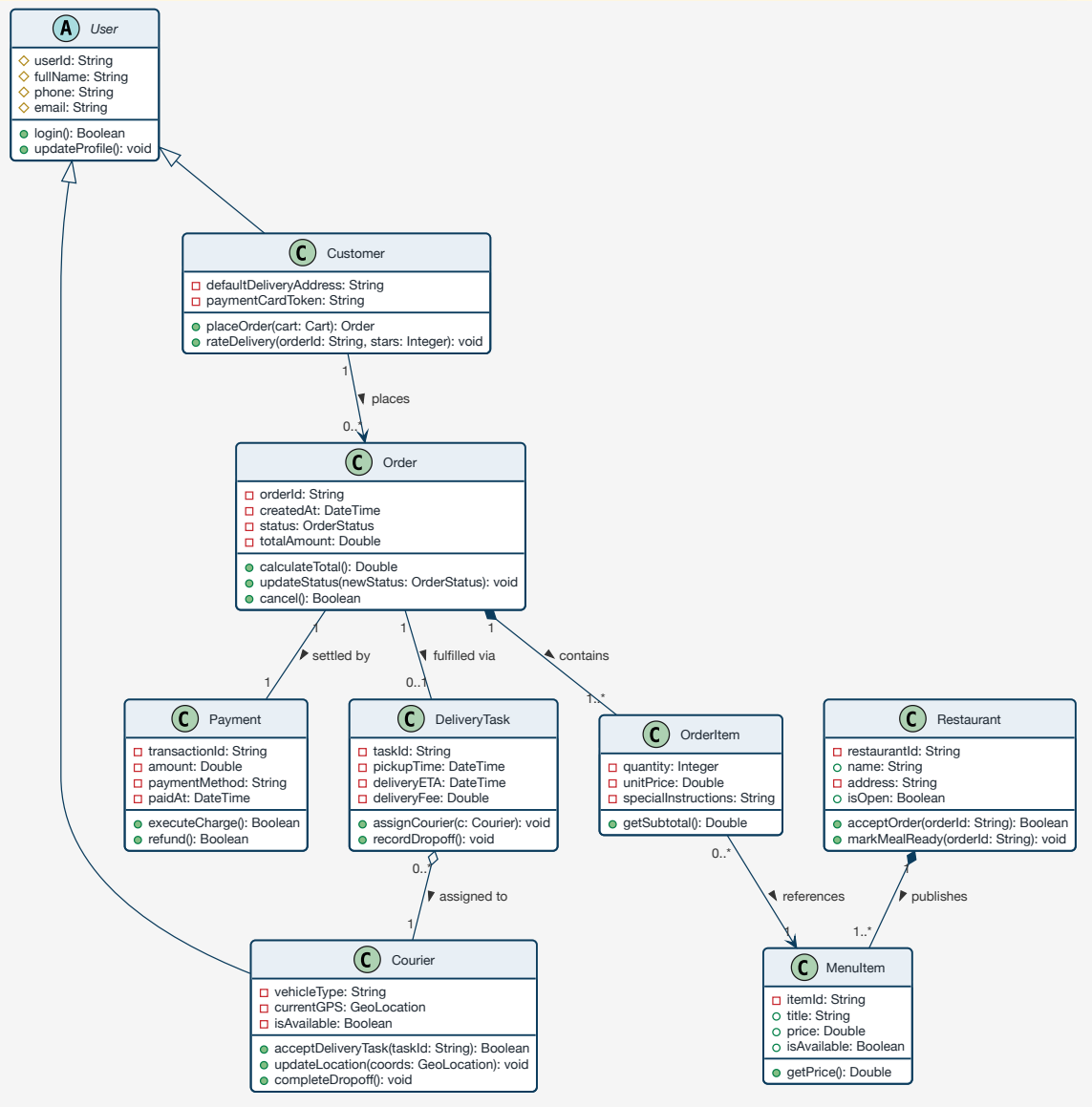
重構生命週期

(聚合 ◇ vs 組合 ◆)

- **聚合 (◇)**：獨立共享生命週期（餐廳與餐點）。
- **組合 (◆)**：級聯銷毀所有權（訂單與明細項目）。

🔴 **工程洞見：** 類別塑模的核心在於定義物件責任邊界與生命週期耦合度——在撰寫任何 SQL 或 OOP 程式碼前，徹底杜絕記憶體洩漏與孤兒資料風險。

美食外送領域：類別圖架構藍圖



領域類別結構與物件生命週期關聯解析

• 泛化繼承階層 (Generalization / Inheritance) :

- `User` 為抽象父類別，定義共用屬性 (`userId`, `phone`, `email`) 與方法 (`login()`)。
- `Customer` 與 `Courier` 特化繼承 `User`，共享帳號身分同時擴充專屬欄位 (如送餐地址 vs. 載具型態與 GPS 座標)。

• 組合 (◼ 實心菱形, Composition) — 強整體-部分擁有關係 :

- `Order "1" *-- "1..*" OrderItem` : `OrderItem` (例如兩份辣味漢堡) 脫離父層 `Order` 便無獨立存在的意義。若訂單被刪除，其下的訂單明細必須連帶被串聯刪除 (Cascade Delete) ！
- `Restaurant "1" *-- "1..*" MenuItem` : 菜單料理嚴格附屬於該發布餐廳。

• 聚合 (◊ 空心菱形, Aggregation) — 弱整體-部分擁有關係 :

- `DeliveryTask "0..*" o-- "1" Courier` : 外送任務分派給外送員，但外送員具備完全獨立的生命週期。外送任務結束或被取消時，外送員依然留在系統中！

• 關聯與價格歷史快照去耦合 :

- `OrderItem` 以 `0..* --> 1` 關聯 `MenuItem`。 `OrderItem` 獨立記錄下單當下的歷史成交單價，未來餐廳調漲漢堡價格時，過去的歷史訂單金額絕不會受到波及。

類別圖標準語法：三格矩形、可見度與關聯重數

- 標準三格類別方塊 (Three-Compartment Box) :
 - 頂層方格：類別名稱，採 `PascalCase`（斜體代表 `abstract` 抽象類別）。
 - 中層方格：屬性欄位： `[可見度] 名稱 : 型別 [= 預設值]` 。
 - 底層方格：操作方法： `[可見度] 名稱 (參數 : 型別) : 回傳型別` 。
- 可見度修飾詞 (Visibility Modifiers，封裝文法) :
 - `+` **Public (公有)**：整個系統任何類別皆可公開存取。
 - `-` **Private (私有)**：嚴格封裝於本類別宣告內部。
 - `#` **Protected (保護)**：僅限本類別與其衍生子類別存取。
 - `~` **Package (套件私有)**：僅限同模組或命名空間內存取。
- 關聯端重數標示 (Multiplicity) :
 - `1`：剛好精確 1 個實例。
 - `0..1`：選用；0 個或 1 個實例。
 - `0..*` (或 `*`)：0 個至多個實例。
 - `1..*`：至少 1 個實例（上不封頂）。

「領域類別模型構築了軟體系統的靜態結構骨幹，為動態互動與生命週期跳轉提供載體。」

靜態結構核心

4.4 領域類別模型

定義領域實體概念、屬性狀態與生命週期耦合結構。

- 實體類別與資料型別
- 關聯重數與基數限制
- 聚合 (◇) 與組合 (◆)

4.3 使用案例模型 (功能合約)

實體資料載體

類別模型提供持久化資料結構與領域實體，支撐使用案例流程中各步驟的資料儲存與狀態維護。

4.5 互動循序圖 (動態交互)

生命線型別與 API

類別模型定義了循序圖中各個生命線物件的所屬型別，並嚴格限制水平訊息呼叫必須對應類別中的合法方法簽章。

4.6 有限狀態機 (生命週期)

內部狀態空間

狀態機深入刻畫類別圖中特定實體類別（如 FoodOrder）的內部狀態變遷；狀態機的屬性即直接對應類別的 status 列舉或 State Pattern。

--- ### 觀念檢核測驗 4 (CCQ 4)

在美食外送類別圖中，為何「訂單 (Order)」與「訂單明細 (OrderItem)」之間採用組合 (◆)，而「外送任務 (DeliveryTask)」與「外送員 (Courier)」之間卻採用聚合 (◇)？

- **A.** 訂單明細可脫離訂單獨立存在；但外送員失去任務就無法存活。
- **B.** 訂單明細隨訂單銷毀而連帶消失；外送員則具備完全獨立的生命週期。
- **C.** 組合代表類別層級的繼承；聚合代表執行時的方法調用。



4.5 互動塑模與循序圖

"Interaction modeling shows how objects collaborate over time to fulfill the promise of a use case."

(互動塑模展示了物件如何在時間維度上協同運作，以實現使用案例的承諾。)

”

循序圖塑模：核心定義與工程價值

「互動塑模揭示了物件實例如何在執行時序中緊密協同，共同履行使用案例的行為承諾。」

”

何謂循序圖模型 (Definition & Abstraction)

- **時間順序訊息流動**：二維動態互動模型，水平軸排列參與物件生命線，垂直軸由上而下代表時序推移。
- **顯性化訊息語意**：清楚呈現同步呼叫 (Synchronous)、非同步訊息 (Asynchronous)、回傳值與物件啟動條長度。
- **結構化控制邏輯**：透過標準化複合片段 (Combined Fragments 如 `alt`、`opt`、`loop`) 表達條件分支與迴圈。

為何至關重要 (Engineering Value)

- **揭示動態執行協同機制**：類別圖只呈現「誰認識誰」，循序圖明確解答「誰在何時呼叫誰、等待多久、傳遞何種參數」。
- **驗證架構分層與責任分配**：實踐 BCE 穩健性架構模式（邊界、控制、實體），杜絕 UI 與資料庫直接耦合的義大利麵代碼。
- **驗證通訊協定與並發邏輯**：作為微服務分散式呼叫、網路逾時因應與多執行緒並發除錯的精確設計藍圖。

循序圖塑模：四步標準工程流程

「循序圖塑模驗證了靜態類別結構是否具備足夠能力協同合作，順利實現動態業務情境。」

步驟 1

鎖定情境腳本

(From Use Case)

- 選定單一具體情境（正常主情境或特定例外分支）。
- 確定本次互動邊界。



步驟 2

識別 BCE 生命線

(Boundary-Control-Entity)

- 映射參與者、介面邊界、控制器與實體物件。
- 由左至右橫向展開。



步驟 3

繪製時序訊息

(Top-Down Messaging)

- 由上而下標示時間流動。
- 區分同步 (→)、非同步 (->) 與回傳虛線 (-->)。



步驟 4 ★ 控制邏輯

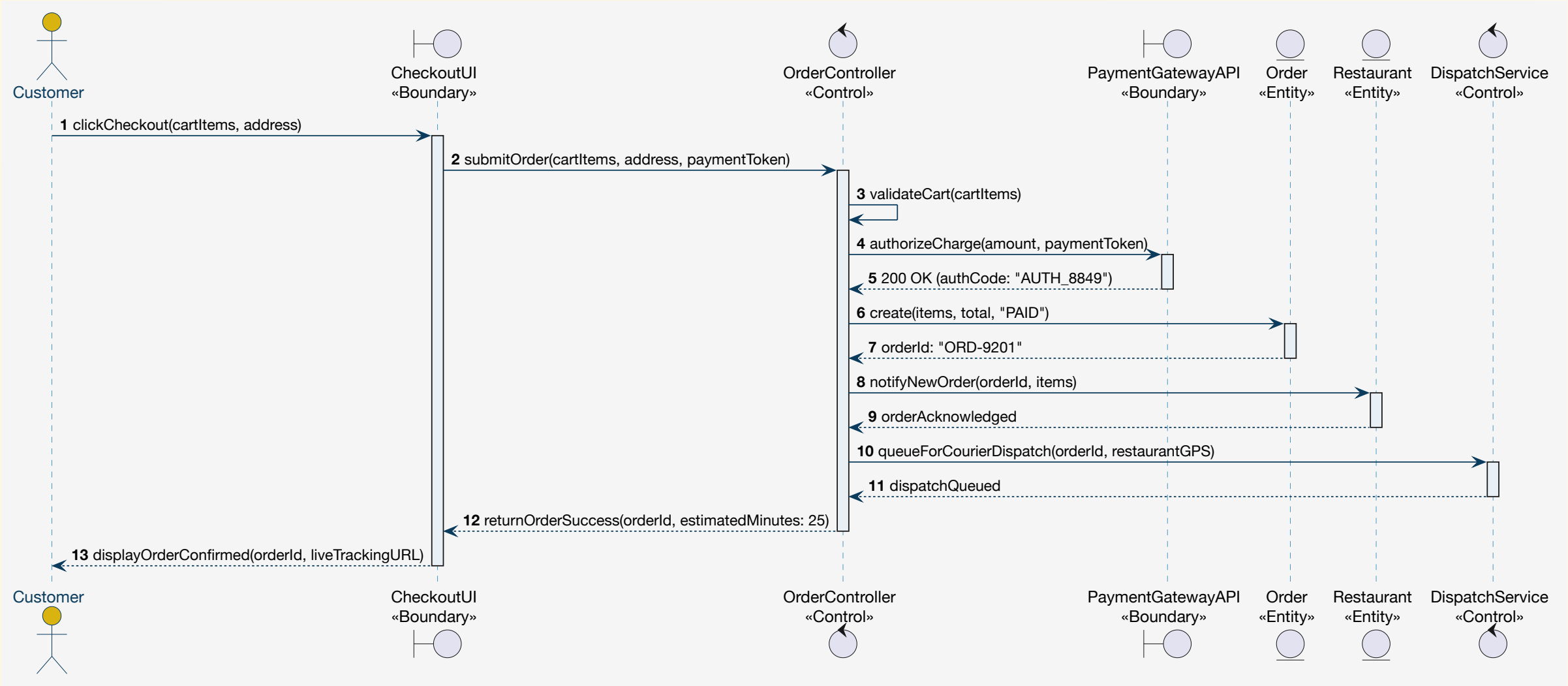
套用複合區塊

(alt / opt / loop)

- **alt:** 互斥條件分支 (if-else)。
- **opt:** 滿足條件時執行的可選行為。
- **loop:** 對集合物件的重複迭代處理。

📌 **工程洞見：**循序圖強制落實 **BCE** 職責分離原則——嚴格禁止 UI 介面邊界直接越權操作資料庫實體，必須經由 Controller 控制層協調調度。

訂單下單與支付流程：UML 循序圖



循序訊息流動與 BCE 架構模式剖析

- 邊界-控制-實體 (BCE, Boundary-Control-Entity) 架構模式：
 - 邊界物件 (`<<Boundary>>`)：負責與外部人類參與者及第三方 API 溝通（如 `CheckoutUI`、`PaymentGatewayAPI`）。
 - 控制物件 (`<<Control>>`)：協調交易流程、業務邏輯演算法與流程分派（如 `OrderController`、`DispatchService`）。
 - 實體物件 (`<<Entity>>`)：封裝持久化儲存的領域狀態與業務核心資料（如 `Order`、`Restaurant`）。
- BCE 架構不可動搖的黃金鐵律：
 - 外部使用者與 UI 邊界物件絕不可直接碰觸或存取 Entity 資料實體物件！
 - 呼叫流向必須嚴格遵守：**Actor** → **Boundary** → **Control** → **Entity**。
 - 原因：徹底實現介面與資料綱要的解耦。未來資料庫欄位或 ORM 變更時，UI 畫面完全不受波及。

循序圖符號與語意指引

視覺符號	語意符號名稱	精確技術語義
垂直虛線	生命線 (Lifeline)	代表該物件實例在記憶體中隨時間推移的存續狀態。
狹長垂直矩形	啟用條 (Activation Bar)	代表該物件實例正在 CPU 上積極執行程式碼運算的時間區間。
實線 + 實心箭頭	同步呼叫 (Synchronous Call)	阻斷性呼叫 (Blocking)；呼叫端暫停等待回傳結果後方能繼續。
實線 + 開放箭頭	非同步訊息 (Asynchronous Call)	非阻斷性訊息 (Non-blocking)；呼叫端送出訊息後立即繼續執行。
虛線 + 開放箭頭	回傳訊息 (Return Message)	將運算結果或資料實例明確回傳給原始呼叫者。
自我迴圈箭頭	自身調用 (Self-Invocation)	物件呼叫自身的私有方法（例如 <code>validateCart()</code> ）。

 **幾何黃金法則：** 時間沿著垂直軸嚴格向下推進。任何訊息箭頭絕對不可向上逆行！

”

「循序圖扮演了動態轉譯的橋樑，將靜態的需求文字與類別定義，化為可執行的訊息調度流。」

動態互動核心

4.5 互動循序圖

跨物件實例的時間序列訊息交換與控制轉移。

- BCE 生命線與啟動條
- 同步與非同步呼叫
- 複合區塊 (alt / loop)

4.3 使用案例模型 (功能需求)

場景動態實現

直接實現使用案例規格書中的文字對話步驟，證明整個系統架構能端到端滿足使用者的業務目標。

4.4 領域類別模型 (結構定義)

方法合法性檢核

每一條水平訊息皆直接檢驗或反哺接收類別的方法簽章 (Method Signatures)，確保職責分配合理。

4.6 有限狀態機 (生命週期)

觸發事件派遣

抵達實體生命線的特定訊息（如 processPayment()），正是激發狀態機產生狀態躍遷的事件觸發源。

--- ### 觀念檢核測驗 5 (CCQ 5)

在採用 **邊界-控制-實體 (BCE)** 架構設計的 UML 循序圖中，當使用者於 `CheckoutUI` 點擊結帳送出時，該請求應該直接交由哪一個物件接收處理？

- **A.** 直接交給 `Order` 實體物件，以便第一時間將訂單寫入資料庫。
- **B.** 直接交給 `PaymentGatewayAPI` 邊界物件，以便立刻進行信用卡扣款。
- **C.** 交給 `OrderController` 控制物件，以統籌驗證業務規則與協調交易。
- **D.** 直接交給 `Restaurant` 實體物件，以確認廚房產能是否充足。



4.6 行為塑模與有限狀態機模型

"A system in dynamic execution is defined by the states it occupies and the events that trigger transitions."

(動態執行中的系統，是由其所處的狀態以及觸發狀態轉換的事件所定義。)

”

狀態機塑模：核心定義與工程價值

「運作中的反應型系統，取決於其所處的離散狀態，以及激發狀態合法躍遷的事件驅動機制。」

”

何謂狀態機模型 (Definition & Abstraction)

- **離散事件驅動模型**：源自 David Harel 的 Statecharts 理論，刻畫單一反應型實體在整個生命週期中如何回應外部事件。
- **狀態分類學**：包含初態 (Initial)、終態 (Final) 及滿足特定商業不變量的穩定運作狀態。
- **轉換語法嚴謹規範**：透過 `觸發事件 [守衛條件] / 動作效果` 形式化宣告狀態躍遷機制。

為何至關重要 (Engineering Value)

- **徹底消除非法業務狀態**：從數學與架構層面杜絕「未付款卻已出貨」、「已取消訂單卻重複退款」等重大業務漏洞。
- **馴服非同步事件的複雜度**：在電商訂單、IoT 設備與長時間工作流中，提供具備確定性與可預測性的可靠行為保障。
- **確立明確的執行防衛線**：嚴格的守衛條件 `[Guard]` 保證狀態躍遷只在業務不變量完全成立時才被放行。

狀態機塑模：四步標準工程流程

「狀態機塑模從數學上杜絕了系統非法狀態的發生，並能有效駕馭非同步事件的複雜度。」

步驟 1

鎖定核心實體

(Stateful Lifecycle)

- 挑選具備多階段生命週期的關鍵類別（如 FoodOrder）。
- 排除無狀態實體。



步驟 2

列舉合法狀態

(Stable Conditions)

- 盤點所有穩定靜止狀態：草稿、已付款、製餐中、已送達。
- 標定起始與終止端點。



步驟 3

繪製事件跳轉

(Event Triggers)

- 繪製狀態間的單向躍遷箭頭路徑。
- 標註觸發躍遷的外部業務事件。



步驟 4 ★ 躍遷文法

守衛與動作效果

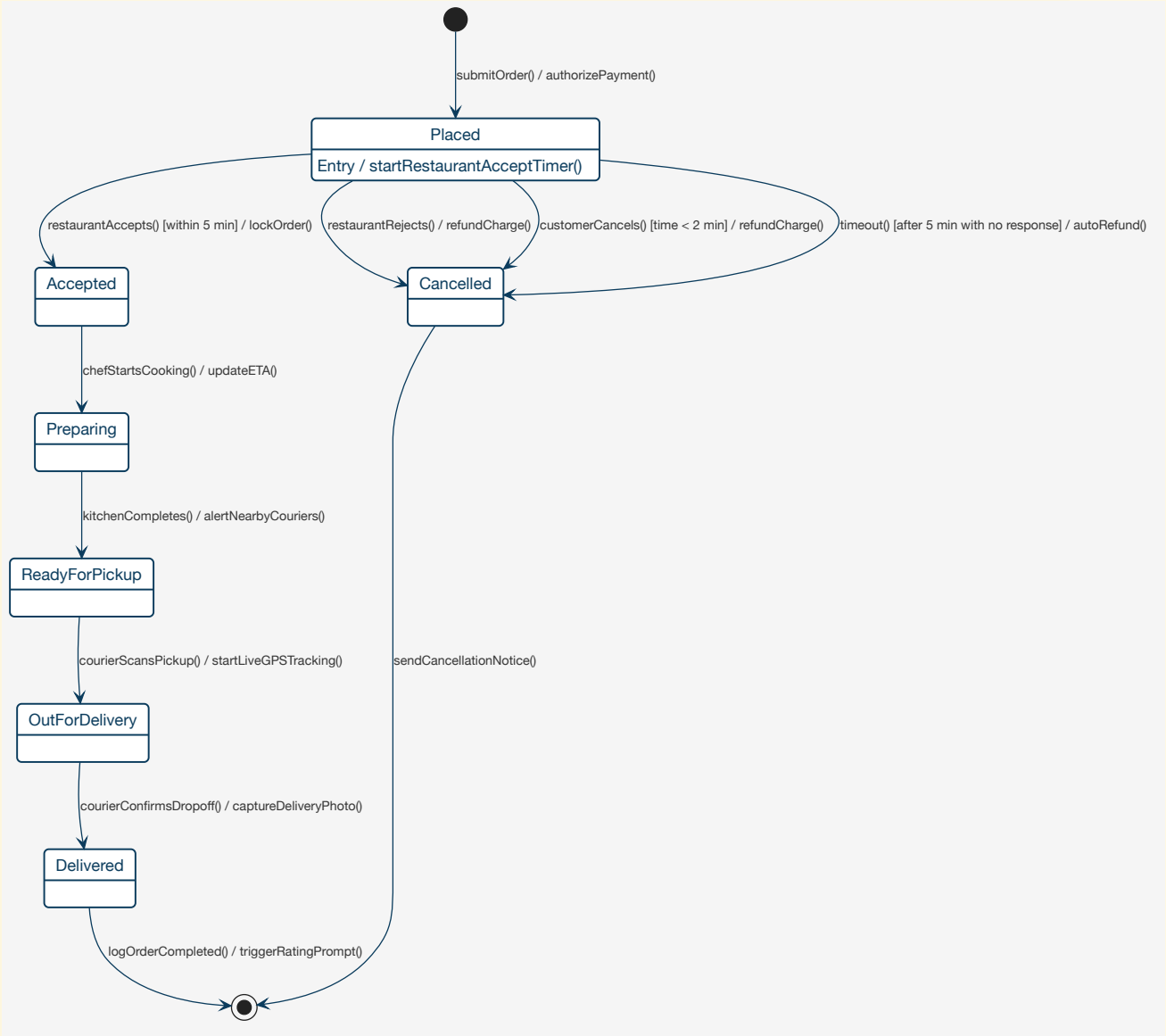
([guard] / action)

- **[guard]**: 必須滿足的布林條件門檻。
- **/ action**: 躍遷時執行的原子副作用。
- 標準語法：

```
event [guard] / action。
```

📌 **工程洞見：** 狀態機在數學邏輯上消滅了非法狀態（如未付款直接出貨），將混亂的巢狀 `if-else` 與 `Flag` 標記轉化為嚴謹可驗證的狀態模式。

訂單生命週期：UML 狀態機圖 (State Machine Diagram)



狀態圖運作原理與轉換文法解析

- 有限狀態機 (FSM) 核心原理：

- 在任何單一運行瞬間，一個 `Order` 物件實例恰好且只能處於一個離散狀態。
- 訂單對外界事件的反應，完全取決於其當前所處的活躍狀態。

- 標準狀態轉換標籤文法 (Formal Label Grammar)：

觸發事件 (Event) [守衛條件 (Guard)] / 執行動作 (Action)

- 觸發事件 (Trigger Event)：刺激狀態轉移的外部或內部事件（如 `chefStartsCooking()` , `courierScansPickup()`）。
- 守衛條件 ([...] Guard Condition)：必須計算為 `true` 轉換才會被放行的布林條件（如 `[within 5 min]` , `[time < 2 min]`）。
- 執行動作 (/ ... Action Effect)：狀態轉移瞬間原子化執行的計算動作（如 `/ refundCharge()` , `/ startLiveGPSTracking()`）。

- 狀態進入動作 (State Entry Actions)：

- 進入特定狀態時自動執行的動作（例如：`Placed: Entry / startRestaurantAcceptTimer()`）。

生命週期守護者

4.6 有限狀態機

複雜領域實體隨時間與事件演進的離散生命週期規則。

- 合法穩定靜止狀態
- 外部事件觸發與信號
- 守衛條件與動作效果

4.3 使用案例模型 (功能合約)

前置條件守衛門檻

實體的當前狀態直接作為使用案例能否被觸發執行的前置守衛門檻（如只有在已付款或備餐中狀態，才允許執行「取消訂單」案例）。

4.4 領域類別模型 (結構定義)

狀態模式實作

狀態機深入描繪類別圖中特定實體類別的動態演變，直接對應 OOP 中的 GoF 狀態模式 (State Pattern) 或列舉變數。

4.5 互動循序圖 (動態交互)

事件來源與動作效果

循序圖中傳入實體生命線的訊息激發了狀態機的狀態躍遷；而躍遷伴隨的動作效果則體現為循序圖後續的呼叫返回。

--- ### 觀念檢核測驗 6 (CCQ 6)

在訂單生命週期狀態機中，標籤

`customerCancels()` `[time < 2 min]` / `refundCharge()` 各自代表何種技術語

義？

- **A.** `customerCancels()` 為守衛條件； `[time < 2 min]` 為觸發事件；
`refundCharge()` 為目標狀態。
- **B.** `customerCancels()` 為觸發事件； `[time < 2 min]` 為布林守衛條件；
`refundCharge()` 為轉換執行動作。
- **C.** `customerCancels()` 為目標類別； `[time < 2 min]` 為呼叫方法；
`refundCharge()` 為傳回型別。



4.7 AI 輔助系統塑模

"AI is a visual copilot that eliminates diagramming friction, but human architects must ensure semantic correctness."

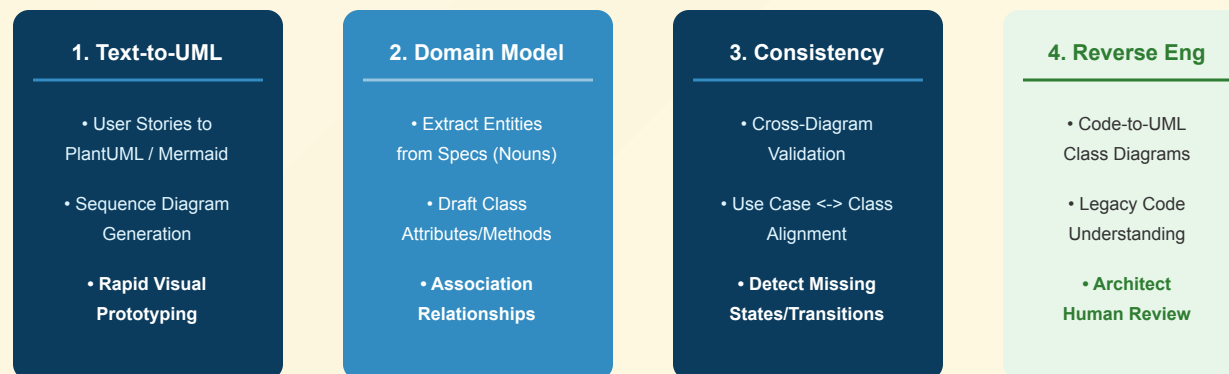
(AI 是一具消除繪圖阻力的視覺副駕駛，但人類架構師必須確保語意的終極正確性。)

”

AI 在系統塑模中的角色：視覺副駕駛 (Visual Copilot)

- **宣告式繪圖革命 (Text-to-Diagram Revolution)：**
 - 大型語言模型能直接將非結構化軟體需求轉譯為宣告式文字標記圖表（如 PlantUML、Mermaid.js、Graphviz）。
 - 在數秒內架起自然語言使用者故事與正式圖形架構模型之間的橋樑。

How AI Assists System Modeling (UML & Diagramming Copilot)



- **生產力的大幅躍升：** 徹底消除惱人的手動排版微調，讓軟體架構師能全神貫注於架構邏輯推理，而非繪圖軟體排版。

AI 在系統塑模中的 4 大核心應用

- **1. 文字直接生成 UML (Text-to-UML Generation) :**
 - 直接根據敏捷使用者故事與 Given-When-Then 驗收準則，自動生成循序圖、類別圖與活動圖標記。
- **2. 領域實體智能萃取 (Domain Entity Extraction) :**
 - 深度語意分析規格需求書，智慧提煉領域名詞（潛在類別、屬性）與動詞（方法操作、關聯）。
- **3. 跨圖表一致性驗證 (Cross-Diagram Consistency Validation) :**
 - 比對檢查使用案例參與者、類別圖與循序圖生命線，偵測命名衝突或未被實作的方法呼叫。
- **4. 程式碼逆向工程塑模 (Code-to-Model Reverse Engineering) :**
 - 讀取現有專案程式碼庫（Java/TypeScript/Python），自動逆向產出類別階層與相依圖，加速新進人員上手。

人機協同防護：AI 塑模風險與最佳實踐

- 未經審查即盲信 AI 塑模的潛在風險：
 - 虛構關聯 (Hallucinated Associations)：幻覺捏造出不存在於業務真實世界中的繼承或組合關聯。
 - 架構過度膨脹 (Architectural Bloat)：硬塞不必要的設計模式（如動輒套用十層工廠裝飾者），違背簡潔原則。
 - 幽靈生命線 (Ghost Lifelines)：在循序圖中憑空捏造不存在的微服務 API 或網路端點。
- 不可動搖的軟體工程黃金守則：

「AI 負責起草繪圖；人類架構師負責檢驗語意！」

(AI Drafts the Diagram; The Human Architect Validates the Semantics!)

軟體架構師必須以批判性工程眼光審視 AI 生成的模型，確保其忠實反映領域真相與架構邊界。

”

觀念檢核測驗 7 (CCQ 7)

當軟體工程團隊導入生成式 AI 輔助自動化產生 UML 圖表時，軟體工程師 / 架構師最核心的職責為何？

- **A.** 拒絕使用任何 AI 工具，堅持純手動編寫每一行繪圖標記語法。
- **B.** 嚴格驗證領域業務語意正確性、結構約束條件與架構簡潔度。
- **C.** 全面廢除傳統程式碼審查（Code Review）與架構設計評審會議。
- **D.** 無條件全盤接受 AI 所生成的所有類別關聯與微服務生命線。



4.8 核心複習與統整

"Models are the lingua franca of software engineering."

(模型，是軟體工程世界的通用語彙。)

”

核心觀念統整：互動填空小測驗

測試你對本章核心概念的掌握度：

1. 在 Rational Software 整合並催生 UML 的「三巨頭」為 Grady Booch、Jim Rumbaugh 與 。
2. 圖用以劃定內部軟體系統與外部夥伴服務雲端之間的營運邊界。
3. 在使用案例塑模中，多個案例必定強制共用的子流程是透過 關係抽離。
4. 將系統劃分為使用者介面、商業邏輯協調與持久化實體的經典架構模式是 。
5. 在 UML 類別圖中，實心黑菱形 () 代表 ，子物件無法脫離父物件獨立存在。
6. 在 UML 類別圖中，空心白菱形 () 代表 ，子物件具備完全獨立的生命週期。
7. UML 狀態機圖之狀態轉移標籤嚴格遵循標準語法：觸發事件 [] / 執行動作。

經典文獻與延伸閱讀 (References)

- 奠基教科書與國際標準規範：

- Sommerville, I. (2016). *Software Engineering* (10th ed.). Chapter 5: System Modeling. Pearson.
- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.
- Fowler, M. (2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.
- Cockburn, A. (2000). *Writing Effective Use Cases*. Addison-Wesley.
- Object Management Group (OMG). (2017). *OMG Unified Modeling Language (OMG UML) Specification*, Version 2.5.1.

- 現代宣告式繪圖標準與 AI 塑模工具：

- PlantUML 開源標準官方網站：plantuml.com
- Mermaid.js JavaScript 宣告式繪圖文件庫：mermaid.js.org